PRACTICAL
REAL-WORLD
BEGINNER FRIENDLY

VERIQTA

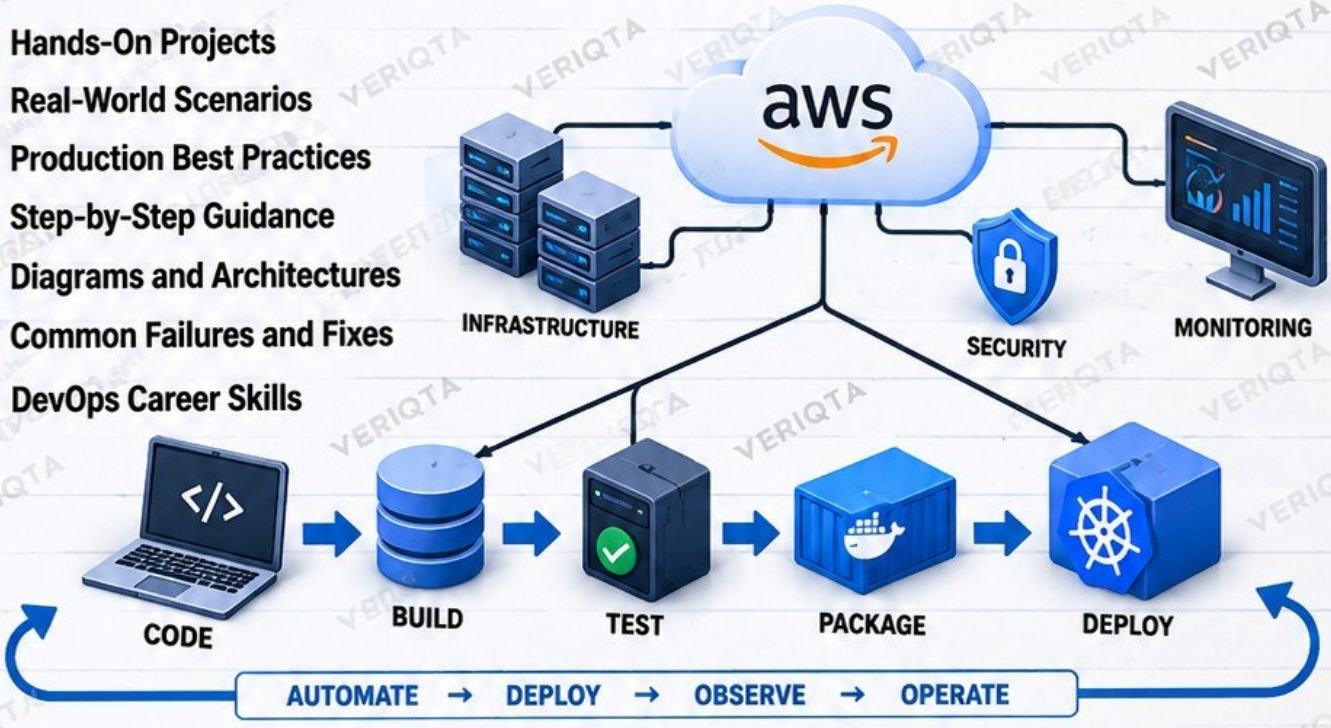
LEARN
BUILD
DEPLOY
GROW

20 PRODUCTION-GRADE DEVOPS PROJECTS

A PRACTICAL, STEP-BY-STEP CURRICULUM TO BUILD REAL-WORLD DEVOPS SKILLS

INFRASTRUCTURE | AUTOMATION | CI/CD | CONTAINERS | KUBERNETES
CLOUD | MONITORING | SECURITY | OPERATIONS | REAL PROJECTS

- Hands-On Projects
- Real-World Scenarios
- Production Best Practices
- Step-by-Step Guidance
- Diagrams and Architectures
- Common Failures and Fixes
- DevOps Career Skills



Linux · aws · Terraform · Ansible · Docker · GitHub · Jenkins · Kubernetes · Argo CD · Prometheus
Grafana · SonarQube · JFrog · Nginx · MySQL · PostgreSQL · Redis · Vault · Helm · And More!

Real Skills
Real Systems
Real Opportunities

From Beginner to Production

Same Tools
Real Engineers Use
Same Skills
Real Companies Need

VERIQTA

LEARN TODAY. BUILD TOMORROW.

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 1 OF 20

LEARN
BUILD
DEPLOY
GROW

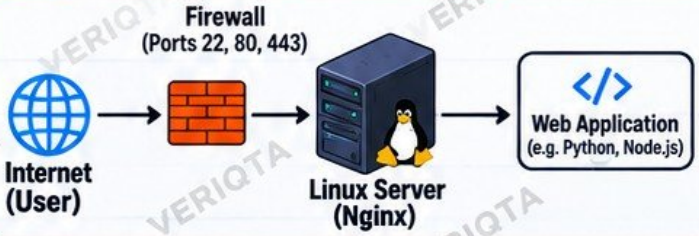
PRODUCTION LINUX WEB SERVER

Build and harden a Linux application server.

PROJECT OVERVIEW

In this project, you will set up and secure a Linux server, deploy a web application and configure Nginx as a reverse proxy. You will also manage services with systemd, configure the firewall, monitor logs and health, and troubleshoot a real failure scenario.

ARCHITECTURE DIAGRAM



KEY LEARNING OBJECTIVES

- Users, groups, SSH keys and permissions
- systemd service management
- Nginx reverse proxy
- Firewall configuration (UFW)
- Logs and health checks
- Troubleshoot a service that will not start
- Production lesson: secure server administration

TECHNOLOGIES USED

Linux (Ubuntu) · Nginx · systemd · UFW (Firewall) · Logs (journalctl)

WHAT YOU WILL BUILD

- A secure Linux server with a non-root user
- SSH key-based authentication
- A web application served through Nginx
- A systemd service for your application
- Firewall rules to allow only required traffic
- Health checks to verify the application is running

IMPLEMENTATION STEPS

1. Create users, groups and SSH key access
2. Secure file permissions (chmod, chown)
3. Install and configure Nginx as a reverse proxy
4. Create a systemd service for your application
5. Configure the UFW firewall
6. Enable logging and monitor logs
7. Set up basic health checks
8. Test, harden and document your setup

INTENTIONAL FAILURE LAB

Simulate a service that will not start.

- Modify the service file with an incorrect path
- Start the service and observe the failure
- Use journalctl, systemctl status and logs to find the cause
- Fix the configuration and verify the service starts

PRODUCTION AWARENESS

- Use least privilege and disable password login
- Keep your server updated
- Allow only necessary ports
- Monitor logs regularly
- Use clear service names and documentation
- Always verify changes after configuration

VERIFICATION

1. Access your application via browser
2. Check service status: systemctl status <service>
3. Confirm Nginx is proxying requests
4. Verify firewall rules: sudo ufw status
5. Check logs for errors
6. Confirm health endpoint returns 200 OK

FINAL RESULT

A secure, production-ready Linux web server running your application with Nginx, systemd, firewall, logging and health checks.

Secure Servers Build Careers!

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

SAME TOOLS
REAL ENGINEERS
REAL IMPACT

LEARN TODAY. BUILD TOMORROW. | VERIQTA

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 2 OF 20

LEARN
BUILD
DEPLOY
GROW

HIGHLY AVAILABLE AWS WEB ARCHITECTURE

Build an application that survives instance failure.

PROJECT OVERVIEW

In this project, you will design and deploy a highly available web application on AWS using a multi-AZ architecture. The application will automatically recover when an EC2 instance fails, ensuring high availability and minimal downtime.

KEY LEARNING OBJECTIVES

- VPC and subnets
- EC2 instances
- Application Load Balancer (ALB)
- Auto Scaling Group (ASG)
- Security Groups
- Route 53 (DNS)
- Multi-AZ design
- Test high availability by terminating an EC2 instance and observing recovery

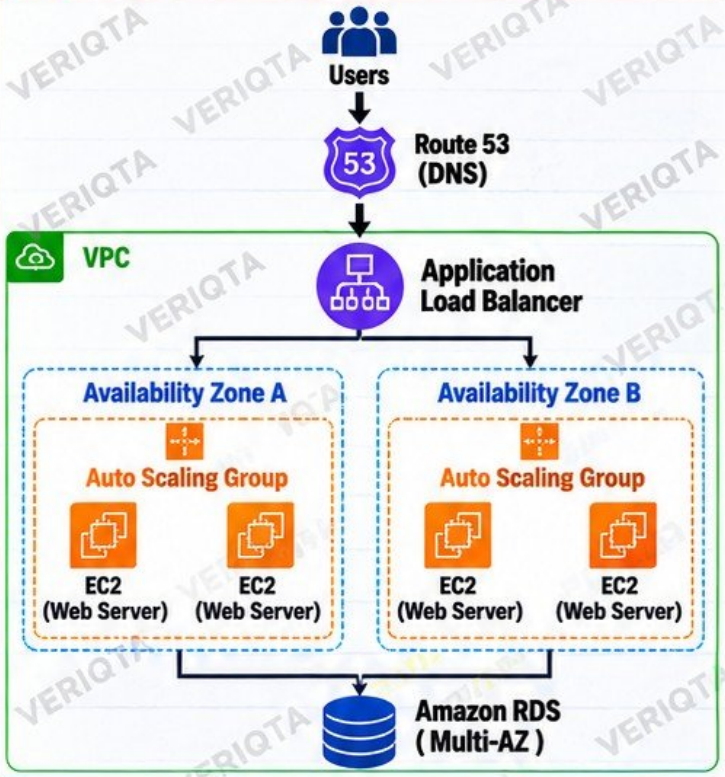
IMPLEMENTATION STEPS

1. Create a VPC with public and private subnets across at least two Availability Zones
2. Configure security groups (least privilege)
3. Launch a Launch Template for your web server
4. Create an Auto Scaling Group (multi-AZ)
5. Set up an Application Load Balancer
6. Deploy a simple web application (e.g. Nginx)
7. Configure Route 53 to point to the ALB
8. Test by terminating an EC2 instance

VERIFICATION

- Application is accessible via Route 53 (domain)
- ALB distributes traffic across instances
- Auto Scaling Group replaces terminated instance
- Application remains available (no downtime)
- 8 Check CloudWatch metrics and ALB health checks

ARCHITECTURE DIAGRAM



TECHNOLOGIES USED

AWS · VPC · EC2 · ALB · ASG · Security Groups · Route 53

INTENTIONAL FAILURE LAB

- Launch the application and confirm it is running
- Terminate one EC2 instance in the Auto Scaling Group
- Observe the Auto Scaling Group replace the instance
- Verify the application remains available through the ALB
- Check logs and metrics during the recovery process

PRODUCTION AWARENESS

- Design for failure, not perfection
- Use multiple Availability Zones
- Use least privilege security groups
- Monitor with CloudWatch
- Set up alerts for instance or ALB failures
- Always test your recovery process

Resilient Systems Power Real Businesss!

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

SAME TOOLS
REAL ENGINEERS
REAL IMPACT

LEARN TODAY. BUILD TOMORROW. | VERIQTA

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 3 OF 20

LEARN
BUILD
DEPLOY
GROW

TERRAFORM AWS INFRASTRUCTURE

Terraform

aws

Rebuild cloud infrastructure as code.

PROJECT OVERVIEW

In this project, you will use Terraform to provision and manage AWS infrastructure as code. You will learn how to use providers, resources, variables, remote state, modules and environment separation to build reproducible and maintainable infrastructure.

KEY LEARNING OBJECTIVES

- Providers and resources
- Variables and outputs
- Remote state
- State locking
- Reusable modules
- Environment separation
- Failure lab: configuration drift
- Production lesson: infrastructure should be reproducible and reviewed

IMPLEMENTATION STEPS

1. Set up Terraform and AWS CLI
2. Configure an AWS provider
3. Create variables and outputs
4. Build core infrastructure (VPC, subnets, EC2, etc.)
5. Configure remote state in S3
6. Enable state locking with DynamoDB
7. Create reusable modules
8. Separate environments (dev, staging, prod)
9. Deploy and validate the infrastructure

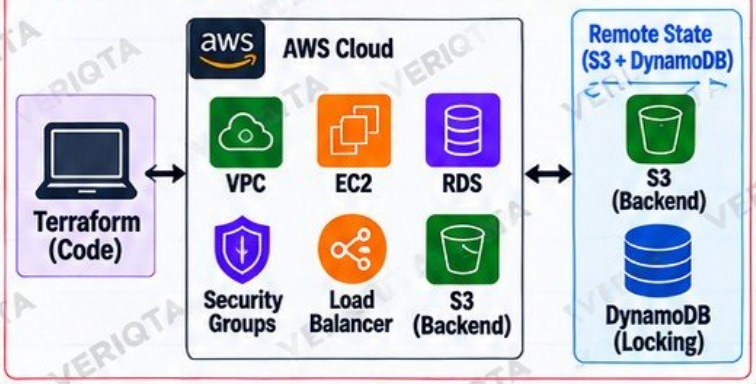
INTENTIONAL FAILURE LAB

- Manually change a resource in the AWS console (e.g. delete a security group rule)
- Run terraform plan and observe the drift
- Use terraform apply to fix the drift
- Discuss why manual changes are a problem

PRODUCTION AWARENESS

- Infrastructure should be reproducible and version controlled
- Avoid manual changes in the cloud console
- Use code review (pull requests) for infrastructure changes
- Document your architecture and maintain a clear structure

ARCHITECTURE DIAGRAM



TECHNOLOGIES USED

Terraform · AWS · S3 (Remote State) · DynamoDB (State Locking) · Terraform Modules

EXAMPLE TERRAFORM CODE

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "veriqta-vpc"
  }
}
```

Infrastructure as Code =
Reproducible
Reliable
Reviewable
Scalable

VERIFICATION

- Run terraform plan (no unexpected changes)
- Confirm resources are created in AWS
- Check remote state is stored in S3
- Verify state locking works with DynamoDB
- Test reusable modules
- Validate environment separation (dev, staging, prod)

> SAME INFRASTRUCTURE.
> EVERY TIME. ANY ENVIRONMENT.
> THAT IS REAL DevOps.
> — VERIQTA

CODE DEPLOY REPEAT IMPROVE

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

SAME TOOLS
REAL ENGINEERS
REAL IMPACT

LEARN TODAY. BUILD TOMORROW. | VERIQTA

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 4 OF 20

LEARN
BUILD
DEPLOY
GROW

ANSIBLE

CONFIGURATION MANAGEMENT WITH ANSIBLE

Configure multiple Linux servers consistently.

AUTOMATE
CONFIGURE
SCALE

PROJECT OVERVIEW

In this project, you will use Ansible to configure multiple Linux servers consistently. You will learn how to use inventory, playbooks, roles, templates, variables, handlers and Ansible Vault to automate server configuration at scale.

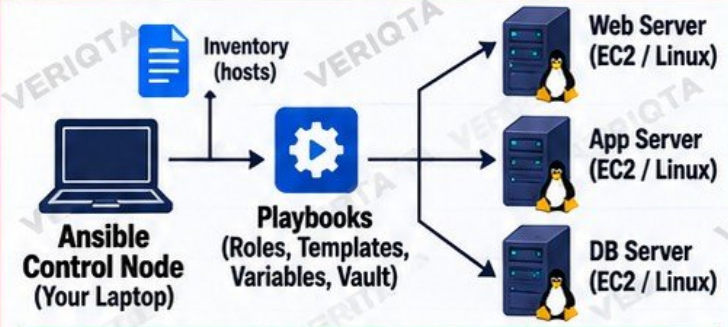
KEY LEARNING OBJECTIVES

- Inventory
- Playbooks
- Roles
- Templates
- Variables
- Handlers
- Idempotency
- Secrets with Ansible Vault
- Failure lab: configuration drift between servers

IMPLEMENTATION STEPS

1. Set up Ansible and configure inventory
2. Create a basic playbook
3. Use variables for flexibility
4. Implement templates (e.g. nginx.conf)
5. Build and use roles
6. Implement handlers (e.g. restart service)
7. Set up Ansible Vault for secrets
8. Configure multiple servers
9. Test idempotency (run playbook multiple times)

ARCHITECTURE DIAGRAM



Write once. Configure many. Keep your servers consistent.

TECHNOLOGIES USED

Ansible · Linux (Ubuntu/CentOS) · YAML · Ansible Vault · AWS (EC2)

EXAMPLE ANSIBLE PLAYBOOK

```
---
- name: Install and start Nginx
  hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present

    - name: Start and enable Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
```

Idempotent
Configurable
Reusable
Secure
Production Ready

INTENTIONAL FAILURE LAB

- Manually change configuration on one server
- Run the playbook and observe the drift being fixed
- Simulate a misconfigured template
- Check which server is out of sync
- Use ansible-playbook --check to preview changes

PRODUCTION AWARENESS

- Keep infrastructure consistent and version controlled
- Use roles and modules for reusability
- Store secrets with Ansible Vault (not in plain text)
- Always test changes in a non-production environment
- Configuration should be reproducible and reviewed

VERIFICATION

- Confirm all servers have the same configuration
- Run ansible-playbook --check (no changes expected)
- Verify services are running
- Use ansible -m ping to check connectivity
- Confirm idempotency (no changes on second run)

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Automate Everything
Stronger Operations!

LEARN TODAY. BUILD TOMORROW. | VERIQTA

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 5 OF 20

LEARN
BUILD
DEPLOY
GROW

PRODUCTION DOCKERIZED APPLICATION

BUILD
PACKAGE
RUN
ANYWHERE

docker

Move an application from server installation to containers.

PROJECT OVERVIEW

In this project, you will containerize a real application (e.g. a Python, Node.js or Nginx app), moving it from a traditional server installation to a Docker container. You will learn how to write a Dockerfile, use multi-stage builds, run containers as non-root, configure environments, use volumes and networks, implement health checks and tag images for versioning.

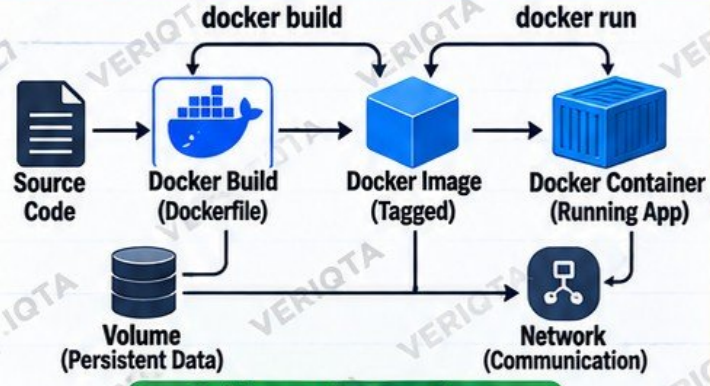
KEY LEARNING OBJECTIVES

- Dockerfile
- Multi-stage builds
- Non-root containers
- Environment configuration
- Volumes
- Networks
- Health checks
- Image tagging
- Failure lab: container repeatedly exits

IMPLEMENTATION STEPS

1. Prepare a simple application (e.g. Python Flask or Node.js)
2. Write a Dockerfile (use multi-stage build)
3. Configure environment variables
4. Run container as non-root user
5. Add a volume for persistent data
6. Create a custom Docker network
7. Implement a health check
8. Build and tag the image (e.g. veriqta/app:v1)
9. Run and test the container

ARCHITECTURE DIAGRAM



Build once. Run anywhere.

TECHNOLOGIES USED

Docker · Python (App Example) · Node.js (App Example) · Nginx (App Example) · Dockerfile

EXAMPLE DOCKERFILE (Python)

```
# Stage 1: Build
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requiremeent.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Stage 2: Run
FROM python:3.11-slim
RUN useradd -m appuser
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11 /usr/local/lib/python3.11
COPY . .
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
   CMD curl -f http://localhost:5000/ || exit 1
CMD ["python", "app.py"]
```

Smaller Images
More Secure
Production Ready

INTENTIONAL FAILURE LAB

- Modify the application to exit immediately
- Build and run the container
- Observe the container repeatedly exiting
- Use docker logs, docker ps and docker inspect
- Identify the root cause and fix the problem

PRODUCTION AWARENESS

- Use minimal base images
- Run containers as non-root
- Keep secrets out of images
- Use health checks for reliability
- Tag images with meaningful versions
- Test containers before production

VERIFICATION

- Container is running and accessible
- Application responds correctly
- Health check passes
- Data persists in the volume
- Container communicates over network
- Image is tagged and can be re-run

Containers Power Modern Applications!

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 6 OF 20

LEARN
BUILD
DEPLOY
GROW

PRIVATE CONTAINER REGISTRY PIPELINE

Build, version, scan, and publish application images.

SECURE IMAGES POWER PRODUCTION

PROJECT OVERVIEW

In this project, you will set up a private container registry using Amazon ECR. You will build a Docker image, tag and version it, scan it for vulnerabilities, and publish it to ECR. You will also learn authentication, immutable image practices, and how to pull and run images securely.

KEY LEARNING OBJECTIVES

- Git (source code)
- Docker (build images)
- Amazon ECR (private registry)
- Image tags and versioning
- Vulnerability scanning
- Authentication (AWS CLI and IAM)
- Immutable artifacts
- Failure lab: image push or pull authentication failure

IMPLEMENTATION STEPS

1. Create a sample application (e.g. Python or Node.js)
2. Push code to a Git repository
3. Build a Docker image
4. Tag the image with a version (e.g. v1, v1.0.0)
5. Configure AWS CLI and IAM permissions
6. Create a private ECR repository
7. Authenticate and push the image to ECR
8. Scan the image for vulnerabilities (Trivy)
9. Verify the image in ECR and pull it to run

PIPELINE DIAGRAM

Build → Scan → Push → Store → Deploy

Secure. Versioned. Scanned. Ready for Production.

TECHNOLOGIES USED

EXAMPLE COMMANDS

```
# Authenticate to ECR
aws ecr get-login-password --region us-east-1 |
docker login --username AWS \
  --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com

# Build and tag image
docker build -t myapp:v1 .
docker tag myapp:v1 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:v1

# Push image
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:v1
```

INTENTIONAL FAILURE LAB

- Use incorrect AWS credentials
- Try pushing without authentication
- Use the wrong region or repository URI
- Observe and analyze the error message
- Fix the issue and successfully push the image

PRODUCTION AWARENESS

- Use immutable image tags (no latest)
- Scan images before pushing
- Restrict access with least privilege IAM
- Keep your registry private
- Track image versions and sources
- Automate scanning in CI pipeline
- Ensure artifacts are reproducible

VERIFICATION

- Image is pushed to ECR successfully
- Image appears in ECR with correct tag
- Vulnerability scan results are available
- Image can be pulled and run
- Access is restricted to authorized users

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

LEARN TODAY. BUILD TOMORROW. | VERIQTA

Build Secure Deploy Repeat!

SAME TOOLS
REAL ENGINEERS
REAL IMPACT

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 7 OF 20

LEARN
BUILD
DEPLOY
GROW

PRODUCTION CI PIPELINE

COMMIT
TEST
SCAN
BUILD
DEPLOY
with CONFIDENCE

Automatically validate every application change.

PROJECT OVERVIEW

In this project, you will set up a GitHub Actions CI pipeline to automatically validate every application change. The pipeline will build the application, run unit tests, lint the code, perform security scanning, create a Docker image, publish artifacts and enforce branch protection rules.

KEY LEARNING OBJECTIVES

- GitHub workflow
- Build process
- Unit tests
- Linting
- Security scanning
- Docker image creation
- Artifact publishing
- Branch protection
- Failure lab: intentionally break a test and trace the failed job

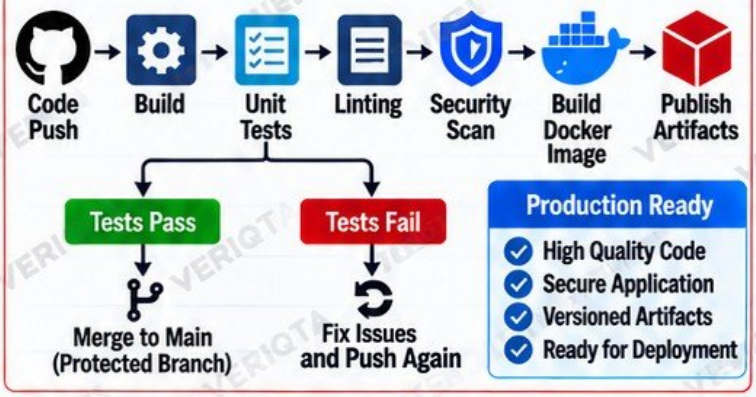
IMPLEMENTATION STEPS

1. Create a sample application (e.g. Python)
2. Add a .github/workflows/ci.yml file
3. Configure build and dependency setup
4. Run unit tests
5. Add linting (e.g. flake8)
6. Add security scanning (e.g. Trivy or SonarQube)
7. Build and tag a Docker image
8. Publish artifacts (e.g. GitHub Packages)
9. Enable branch protection rules

INTENTIONAL FAILURE LAB

- Introduce a failing unit test (e.g. assert False)
- Push the code and observe the failed job
- Check logs to identify the faillre
- Fix the test and push again
- Verify the pipeline passes

CI PIPELINE DIAGRAM



TECHNOLOGIES USED

GitHub Actions · Python (App Example) · Docker · SonarQube (Security Scan) · Artifact Storage (GitHub Packages)

EXAMPLE GITHUB WORKFLOW (Simplified)

```yaml
name: CI Pipeline
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
      - uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirentes.txt
      - name: Run tests
        run: pytest
      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .
      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: build-artifact
          path: dist/
```

Small Changes
Big Impact
Automate
Everything

PRODUCTION AWARENESS

- Automate validation for every change
- Catch issues early before deployment
- Use security scanning in your pipeline
- Enforce branch protection rules
- Store and version build artifacts
- A passing pipeline does not guarantee application health in production

VERIFICATION

- Pipeline runs automatically on push
- Build, test, lint and scan succeed
- Docker image is created and tagged
- Artifacts are published
- Branch protection blocks failing changes
- Fixing the test makes the pipeline pass

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 8 OF 20

LEARN
BUILD
DEPLOY
GROW

COMPLETE CI/CD DEPLOYMENT PIPELINE

CI CD

BUILD FASTER | TEST SAFER | DEPLOY BETTER

Take a commit safely from Git to production.

AUTOMATE
DEPLOY
VERIFY
ROLLBACK
REPEAT

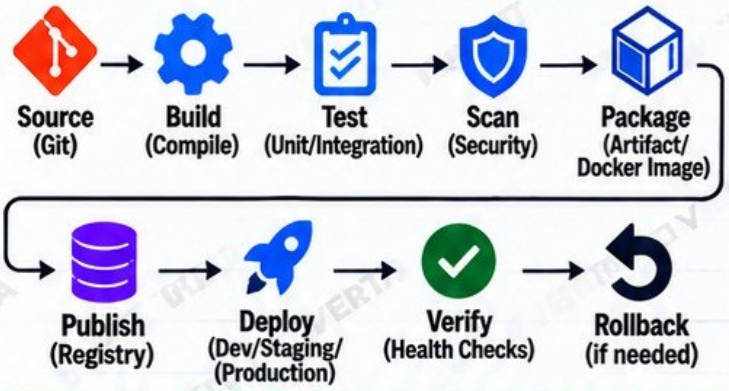
PROJECT OVERVIEW

In this project, you will build a complete CI/CD pipeline that takes a code commit from source control to a production environment. You will learn how to build, test, scan, package, publish, deploy, værify and roll back safely.

KEY LEARNING OBJECTIVES

- Source (Git)
- Build
- Test
- Scan (Security)
- Package
- Publish (Artifact/Image)
- Deploy
- Verify
- Rollback
- Production lesson: deployment success does not automatically mean application health

CI/CD PIPELINE DIAGRAM



Automate. Validate. Deploy with Confidence.

TECHNOLOGIES USED

Git | GitHub Actions | Docker | Container Registry (ECR/Docker Hub) | Kubernetes (or any platform) | AWS (Example)

IMPLEMENTATION STEPS

1. Set up a source repository (Git/GitHub)
2. Configure CI workflow (build, test, scan)
3. Build the application (e.g. Maven, Node.js, Python)
4. Run unit and integration tests
5. Add security scanning (e.g. Trivy, SonarQube)
6. Package the application (Docker image or artifact)
7. Publish to a container registry (e.g. Amazon ECR)
8. Deploy to staging and production (e.g. Kubernetes)
9. Verify deployment (health checks, logs, metrics)
10. Implement rollback strategy

EXAMPLE PIPELINE (GitHub Actions)

```yaml
name: CI/CD Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build-test-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: echo "Building application..."
      - name: Test
        run: echo "Running tests..."
      - name: Build Docker Image
        run: docker build -t myapp:latest .
      - name: Push to Registry
        run: docker push <registry>/myapp:latest
      - name: Deploy
        run: kubectl apply -f k8s/
      - run: echo "Running health checks..."
```

CODE
TEST
SCAN
DEPLOY
VERIFY
IMPROVE

INTENTIONAL FAILURE LAB

- Break a unit test and push the code
- Observe the failed job in the pipeline
- Check logs and identify the failure
- Fix the issue and push again
- Verify the pipeline succeeds

PRODUCTION AWARENESS

- A successful deployment does not always mean the application is healthy
- Implement proper health checks
- Monitor logs, metrics and alerts
- Always have a rollback plan
- Test your rollback process
- Communicate changes to stakeholders

VERIFICATION

- Pipeline runs successfully from commit to production
- Application is accessible and healthy
- All tests and security scans pass
- Artifacts are published correctly
- Deployment can be rolled back
- Monitor and confirm application health

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 9 OF 20

LEARN
BUILD
DEPLOY
GROW

JENKINS PRODUCTION CI/CD PLATFORM

Jenkins

Build a self-managed automation platform.

AUTOMATE
INTEGRATE
DEPLOY
AT SCALE

PROJECT OVERVIEW

In this project, you will set up a self-managed Jenkins CI/CD platform. You will configure a Jenkins controller and agents, manage credentials, create pipelines as code, use webhooks, handle artifacts, build Docker images, and implement deployment stages. You will also troubleshoot common production issues such as failed agents or expired credentials.

KEY LEARNING OBJECTIVES

- Jenkins controller
- Agents
- Credentials
- Pipeline as Code
- Webhooks
- Artifact handling
- Docker builds
- Deployment stages
- Failure lab: failed agent or expired credential

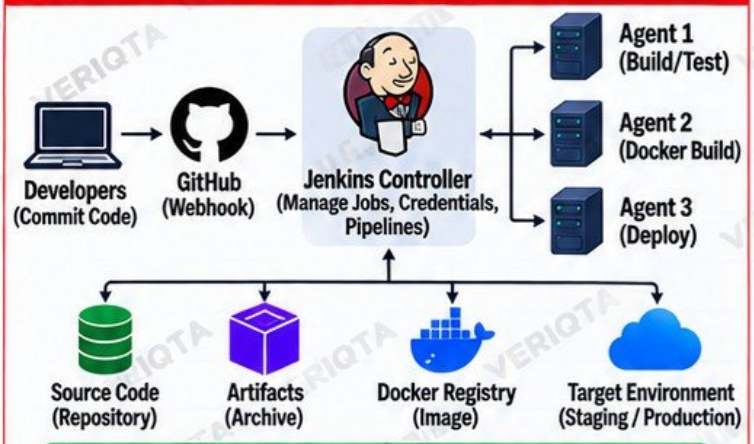
IMPLEMENTATION STEPS

1. Install and configure Jenkins (controller)
2. Set up Jenkins agents (static or dynamic)
3. Configure credentials (Git, Docker, cloud, etc.)
4. Create a pipeline (Jenkinsfile)
5. Set up webhooks from GitHub
6. Build the application (e.g. Maven or Node.js)
7. Build and push a Docker image
8. Handle artifacts (archive and store)
9. Add deployment stages (e.g. staging and production)
10. Test failure scenarios (agent down or expired credential)

INTENTIONAL FAILURE LAB

- Stop a Jenkins agent and run the pipeline
- Use an expired or incorrect credential
- Observe the failed job and error messages
- Troubleshoot and fix the issue
- Rerun the pipeline and verify success

JENKINS ARCHITECTURE DIAGRAM



TECHNOLOGIES USED

Jenkins · GitHub (Webhooks) · Docker · Maven · Linux (Agents) · Cloud (AWS / Other)

EXAMPLE JENKINS PIPELINE (Jenkinsfile)

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building application...'
                sh 'mvn clean package'
            }
        }
        stage('Docker Build') {
            steps {
                sh 'docker build -t veriqta/app:${BUILD_NUMBER} .'
                sh 'docker push veriqta/app:${BUILD_NUMBER}'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying to production...'
            }
        }
    }
}
```

PIPELINE
AS CODE
REPEATABLE
RELIABLE
SCALABLE

PRODUCTION AWARENESS

- Secure your Jenkins installation
- Use least privilege for credentials
- Monitor Jenkins and agents
- Clean up old builds and artifacts
- Use pipeline as code (Jenkinsfile)
- Plan for high availability and backups
- Test failure scenarios regularly

VERIFICATION

- Pipeline runs successfully from commit
- Build and tests complete
- Docker image is created and published
- Application is deployed to target environment
- Pipeline fails with incorrect credentials
- Agent failure is detected and handled
- Rollback works when needed

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 10 OF 20

LEARN
BUILD
DEPLOY
GROW

KUBERNETES APPLICATION DEPLOYMENT

Move the containerized application into Kubernetes.

SCALE
DEPLOY
MANAGE
REAL APPS
IN KUBERNETES

PROJECT OVERVIEW

In this project, you will deploy a containerized application into Kubernetes. You will learn how to work with pods, deployments, services, ConfigMaps, secrets, namespaces, resource requests and limits, and perform rolling updates. You will also troubleshoot common issues such as CrashLoopBackOff.

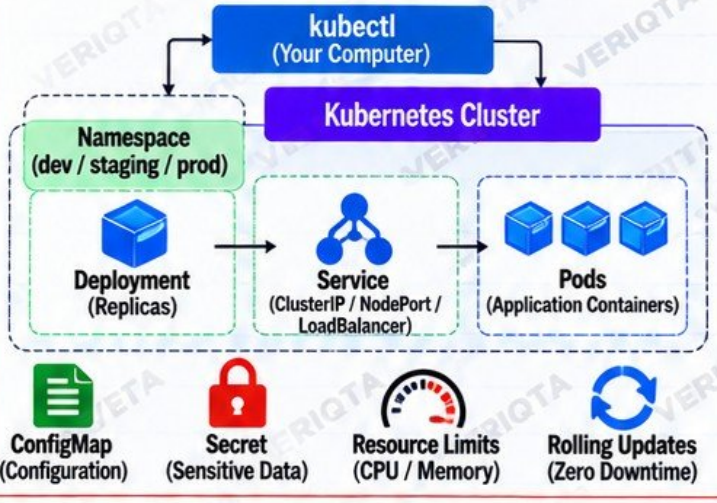
KEY LEARNING OBJECTIVES

- Pods
- Deployments
- Services
- ConfigMaps
- Secrets
- Namespaces
- Resource requests and limits
- Rolling updates
- Failure lab: CrashLoopBackOff

IMPLEMENTATION STEPS

1. Build a containerized application (e.g. Node.js or Python)
2. Create a Kubernetes namespace
3. Create ConfigMap and Secret
4. Deploy the application (Deployment)
5. Expose it using a Service
6. Set resource requests and limits
7. Perform a rolling update
8. Verify the deployment and application access
9. Simulate and fix a failure (CrashLoopBackOff)

KUBERNETES ARCHITECTURE DIAGRAM



TECHNOLOGIES USED

Kubernetes · Docker · Helm (Optional) · YAML · Cloud (AWS / Other)

EXAMPLE DEPLOYMENT MANIFEST (YAML)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-app
  namespace: dev
spec:
  replicas: 3
  selector:
    matchLabels:
      app: veriqta-app
  template:
    metadata:
      labels:
        app: veriqta-app
    spec:
      containers:
      - name: app
        image: veriqta/app:1.0.0
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
```

DEPLOY
SCALE
UPDATE
TROUBLESHOOT
LIKE A PRO

INTENTIONAL FAILURE LAB

- Deploy a pod with a wrong configuration
- Observe CrashLoopBackOff status
- Check pod logs and describe the pod
- Identify the root cause and fix the issue
- Verify the application is running

PRODUCTION AWARENESS

- Always use resource requests and limits
- Store sensitive data in Secrets
- Use namespaces for environment separation
- Prefer rolling updates for zero downtime
- Monitor application health and pod status
- Set liveness and readiness probes
- Plan for rollback in case of issues

VERIFICATION

- Pods are running and healthy
- Service is accessible
- ConfigMaps and Secrets are correctly used
- Resource limits are enforced
- Rolling updates work without downtime
- Application recovers after failure
- You can troubleshoot and fix CrashLoopBackOff

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 11 OF 20

LEARN
BUILD
DEPLOY
GROW

aws

Amazon EKS

PRODUCTION KUBERNETES ON AMAZON EKS

Build a managed Kubernetes environment.

MANAGED
SCALABLE
SECURE
PRODUCTION
READY

PROJECT OVERVIEW

In this project, you will set up a production-ready Kubernetes environment on Amazon EKS. You will learn how to create an EKS cluster, configure worker nodes, set up IAM, design VPC networking, use load balancing, configure storage, implement Kubernetes RBAC, and deploy an application. You will also troubleshoot common issues such as a Pod running but not accessible from the internet.

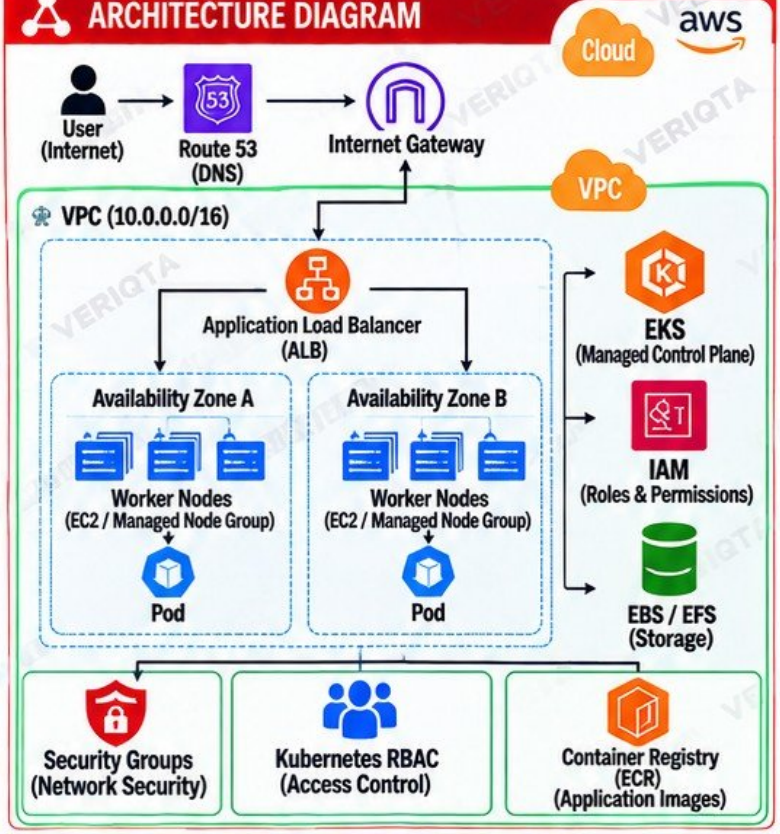
KEY LEARNING OBJECTIVES

- EKS (Managed Kubernetes)
- Worker nodes (Managed Node Groups)
- IAM (Roles and permissions)
- VPC networking (subnets, routes, security groups)
- Load balancing (AWS Load Balancer Controller)
- Storage (EBS and EFS)
- Kubernetes RBAC
- Application deployment
- Failure lab: application Pod runs but cannot receive external traffic

IMPLEMENTATION STEPS

1. Create a VPC with public and private subnets
2. Create an EKS cluster
3. Set up IAM roles and policies
4. Create managed node groups (worker nodes)
5. Configure kubectl and access the cluster
6. Install AWS Load Balancer Controller
7. Configure storage (EBS/EFS)
8. Deploy a sample application (e.g. Nginx)
9. Expose the application using a LoadBalancer or Ingress

ARCHITECTURE DIAGRAM



EXAMPLE DEPLOYMENT COMMANDS

```
# Get EKS cluster info
aws eks describe-cluster --name veriqta-eks

# Update kubeconfig
aws eks update-kubeconfig --name veriqta-eks

# Deploy a sample application
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

# Check resources
kubectl get pods,svc -n veriqta-app

# Get external URL (if using LoadBalancer)
kubectl get svc -n veriqta-app
```

DEPLOY
MONITOR
TROUBLESHOOT
LIKE A
PRODUCTION
ENGINEER

INTENTIONAL FAILURE LAB

- Deploy the application and confirm Pod is running
- Application is not accessible from the internet
- Check service, load balancer, security groups and subnet configuration
- Find the root cause and fix the issue
- Verify external access works

PRODUCTION AWARENESS

- A running Pod does not mean the app is healthy
- Check end-to-end connectivity (DNS, ALB, SG)
- Use least privilege IAM roles
- Monitor cluster and node health
- Use multiple availability zones
- Implement proper logging and monitoring
- Plan for upgrades and disaster recovery

VERIFICATION

- EKS cluster is healthy
- Worker nodes are ready
- Application is deployed and running
- Application is accessible externally
- Storage is working
- RBAC is configured correctly
- You can troubleshoot and fix issues

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 12 OF 20

LEARN
BUILD
DEPLOY
GROW

KUBERNETES INGRESS AND TLS

Expose applications securely.

SECURE
SCALE
RELIABLE
REAL-WORLD
KUBERNETES

PROJECT OVERVIEW

In this project, you will expose applications securely on Kubernetes using Ingress, TLS certificates and the AWS Load Balancer Controller. You will learn how DNS works, configure routing rules, enable HTTPS, perform health checks, and troubleshoot common issues such as certificate or routing failures.

KEY LEARNING OBJECTIVES

- DNS (Domain Name System)
- Ingress
- AWS Load Balancer Controller
- TLS certificates (e.g. ACM, cert-manager)
- HTTPS
- Routing rules (host and path based)
- Health checks
- Failure lab: certificate or routing failure
- Request flow: User → DNS → Load Balancer → Ingress → Service → Pod

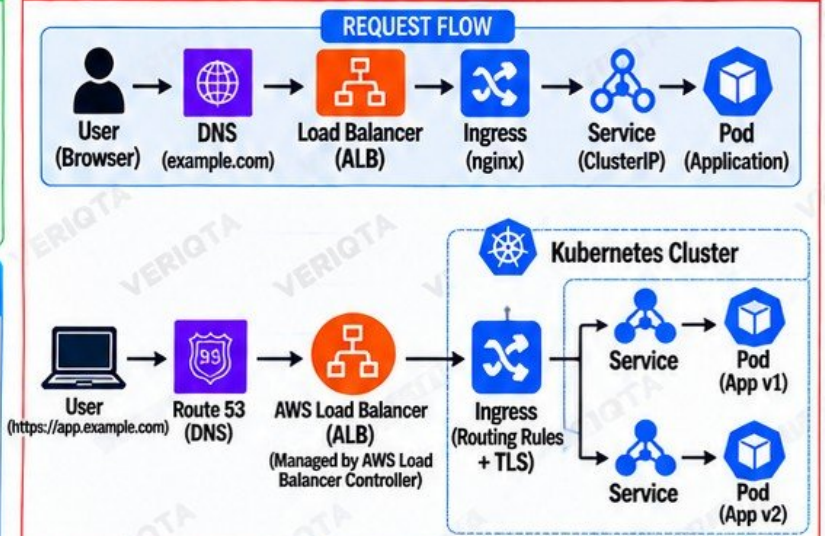
IMPLEMENTATION STEPS

1. Set up a domain name in Route 53 (DNS)
2. Install and configure AWS Load Balancer Controller
3. Create a TLS certificate (ACM or cert-manager)
4. Create an Ingress resource with routing rules
5. Configure services and deploy the application
6. Enable HTTPS and test access
7. Verify health checks and monitoring
8. Simulate and fix a certificate or routing failure

INTENTIONAL FAILURE LAB

- Use an invalid or expired TLS certificate
- Misconfigure DNS record
- Wrong Ingress host or path rule
- ALB health check fails
- Application Pod runs but cannot receive external traffic
- Identify the root cause and fix the issue

ARCHITECTURE DIAGRAM



TLS CERTIFICATE

- AWS Certificate Manager (ACM) or cert-manager
- Automatic certificate renewal
- HTTPS encryption

ROUTING RULES

- Host based routing (e.g. app.example.com)
- Path based routing (e.g. /api, /web)
- Multiple services

HEALTH CHECKS

- ALB health checks
- Ingress health checks
- Kubernetes readiness and liveness probes

EXAMPLE INGRESS RESOURCE (YAML)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: veriqta-app
  namespace: default
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:region:account:certificate/xxxxx
spec:
  tls:
    - hosts:
        - app.example.com
      secretName: app-tls
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: veriqta-app
                port:
                  number: 80
```

SECURE
ROUTE
AUTOMATE
TROUBLESHOOT
LIKE A PRO

PRODUCTION AWARENESS

- Use valid and trusted TLS certificates
- Automate certificate renewal
- Monitor ALB and Ingress logs
- Use health checks and alerts
- Plan for high availability
- Document DNS, routing and TLS setup
- Test failure scenarios regularly

VERIFICATION

- Application is accessible via domain name
- HTTPS is working (valid certificate)
- Routing rules send traffic to the correct service
- Health checks are passing
- Pod is running and serving traffic
- You can troubleshoot and fix certificate or routing failures

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 13 OF 20

LEARN
BUILD
DEPLOY
GROW

GITOPS CONTINUOUS DELIVERY WITH ARGO CD

argo cd

Make Git the source of truth for Kubernetes.

AUTOMATE
DEPLOY
MONITOR
KEEP CONFIG
IN GIT

PROJECT OVERVIEW

In this project, you will implement GitOps continuous delivery using Argo CD. You will make Git the source of truth for Kubernetes by using a separate application repository and deployment repository. You will learn how to manage Kubernetes manifests, configure Argo CD, synchronize applications, detect drift, perform rollbacks, and troubleshoot issues such as manual changes to cluster resources.

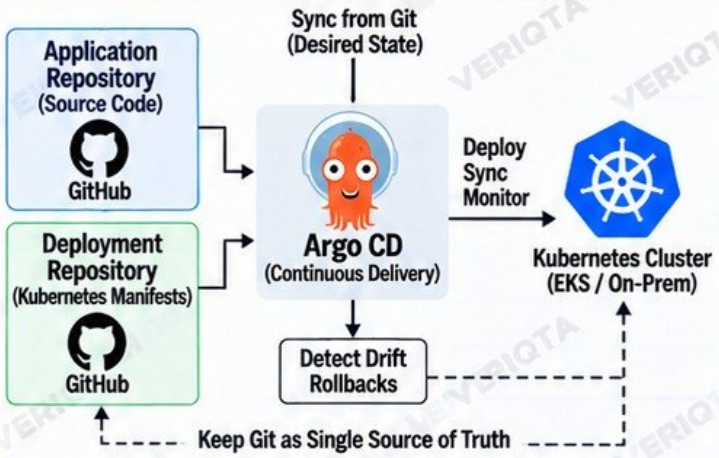
KEY LEARNING OBJECTIVES

- Application repository
- Deployment repository
- Kubernetes manifests
- Argo CD
- Synchronization
- Drift detection
- Rollbacks
- Failure lab: manually change a cluster resource and observe drift

IMPLEMENTATION STEPS

1. Set up a Kubernetes cluster (local or EKS)
2. Install Argo CD
3. Create an application repository (sample app)
4. Create a deployment repository (Kubernetes manifests)
5. Deploy an application using Argo CD
6. Configure synchronization (auto or manual)
7. Make a manual change to a cluster resource
8. Observe drift detection in Argo CD
9. Roll back to the Git version and verify

GITOPS ARCHITECTURE DIAGRAM



User → Git (Code & Manifests) → Argo CD → Kubernetes → Running Application

TECHNOLOGIES USED

GitHub · Argo CD · Kubernetes · YAML · Amazon EKS (or any cluster)

EXAMPLE ARGO CD APPLICATION (YAML)

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: veriqta-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/veriqta/k8s-manifests.git
    targetRevision: main
    path: veriqta-app
  destination:
    server: https://kubernetes.default.svc
    namespace: veriqta-app
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

GIT
AUTOMATES
KUBERNETES
SIMPLE
RELIABLE
SCALABLE

INTENTIONAL FAILURE LAB

- Manually change a Kubernetes resource (e.g. edit replicas)
- Observe the OutOfSync status in Argo CD
- Check the differences between Git and cluster
- Synchronize to restore the desired state
- Verify the application returns to Synced

PRODUCTION AWARENESS

- Keep Git as the single source of truth
- Use separate repos for code and manifests
- Enable automated sync with caution
- Monitor application health and sync status
- Use RBAC to secure Argo CD
- Document changes and access control
- Plan rollbacks and disaster recovery

VERIFICATION

- Application shows Synced in Argo CD
- Kubernetes resources match Git
- Manual changes are detected as OutOfSync
- Rollback restores the Git version
- Application remains healthy after sync
- You can troubleshoot and fix drift issues
- Understand GitOps benefits and limitations

VERIQTA

@veriqta · @veriqta · @veriqta · @veriqta · @veriqta · @veriqta

LEARN TODAY. BUILD TOMORROW. | VERIQTA

GitOps
Real Skills
Real Opportunities!

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 14 OF 20

LEARN
BUILD
DEPLOY
GROW

KUBERNETES DEPLOYMENT STRATEGIES

Deploy without unnecessary downtime.

SAFER DEPLOYMENTS
HAPPIER USERS
STRONGER ENGINEERS

PROJECT OVERVIEW

In this project, you will learn and implement different Kubernetes deployment strategies to deploy applications without unnecessary downtime. You will explore rolling updates, blue-green deployments, canary releases, readiness probes, traffic shifting, automated verification, and rollback criteria. You will also simulate a real failure by releasing a deliberately unhealthy version and observing the impact and recovery process.

KEY LEARNING OBJECTIVES

- Rolling updates
- Blue-green deployments
- Canary releases
- Readiness probes
- Traffic shifting
- Automated verification
- Rollback criteria
- Failure lab: release a deliberately unhealthy version

IMPLEMENTATION STEPS

1. Create a sample application (v1)
2. Deploy using a rolling update
3. Implement blue-green deployment
4. Implement canary release with traffic shifting
5. Configure readiness probes
6. Set up automated verification (health checks)
7. Define rollback criteria
8. Release a deliberately unhealthy version
9. Observe the impact and perform a rollback

DEPLOYMENT STRATEGIES DIAGRAMS

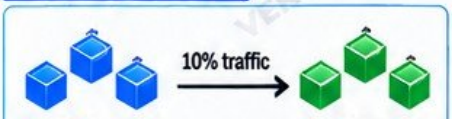
1. ROLLING UPDATE (Zero Downtime)

- Gradual replacement
- No downtime
- Built-in Kubernetes
- Good for most apps

2. BLUE-GREEN DEPLOYMENT

- Two identical environments
- Instant traffic switch
- Easy rollback
- Minimal risk
- Requires more resources

3. CANARY RELEASE



- Release to a small user group
- Monitor metrics and errors
- Gradually increase traffic
- Lower risk
- Good for high-traffic apps

READINESS PROBE EXAMPLE (YAML)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: veriqta-app
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: app
          image: veriqta/app:v1
          ports:
          - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 5
```

HEALTH CHECKS PREVENT DOWNTIME

INTENTIONAL FAILURE LAB

- Deploy a new version with a bug (e.g. /health returns 500)
- Observe how probes mark the pod as unready
- Check rollout status and application behavior
- Verify monitoring and alerts
- Perform a rollback to the previous healthy version
- Confirm the application is restored

PRODUCTION AWARENESS

- Always use readiness and liveness probes
- Automate verification before full rollout
- Define clear rollback criteria
- Monitor error rates and latency
- Use progressive traffic shifting for risky releases
- Test deployment strategies in non-production
- Have a rollback plan and practice it
- Communicate changes to your team

VERIFICATION

- Application remains available
- Rolling update completes successfully
- Blue-green switch works as expected
- Canary release routes traffic correctly
- Readiness probes prevent bad pods
- Monitoring shows healthy metrics
- Rollback restores the previous version
- You can troubleshoot and fix issues

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 15 OF 20

LEARN
BUILD
DEPLOY
GROW

PRODUCTION SECRETS MANAGEMENT

Remove credentials from application code and unsafe configuration.

SECURE CREDENTIALS
SECURE APPS
SECURE INFRASTRUCTURE

PROJECT OVERVIEW

In this project, you will implement production-ready secrets management using AWS Secrets Manager and Kubernetes. You will learn how to remove credentials from application code, use IAM roles and workload identity, inject secrets into applications, implement rotation, and follow the principle of least privilege. You will also simulate a real failure scenario where the application loses access after credential rotation and troubleshoot the issue.

KEY LEARNING OBJECTIVES

- AWS Secrets Manager
- Kubernetes Secrets
- IAM roles
- Workload identity (IRSA)
- Secret injection (environment variables / files)
- Rotation
- Least privilege
- Failure lab: application loses access after credential rotation

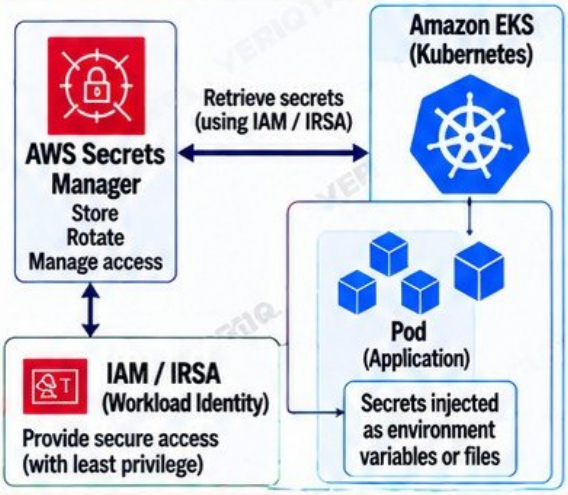
IMPLEMENTATION STEPS

1. Create a secret in AWS Secrets Manager
2. Configure IAM roles and policies (least privilege)
3. Set up workload identity (IRSA) for Kubernetes
4. Inject secrets into a Kubernetes application
5. Implement secret rotation
6. Update application to handle rotated credentials
7. Test the application after rotation
8. Simulate and fix a failure scenario

INTENTIONAL FAILURE LAB

- Rotate the credential in AWS Secrets Manager
- Observe the application losing access
- Check application logs and error messages
- Verify IAM role and permissions
- Update the application or reload the secret
- Confirm the application works again

ARCHITECTURE DIAGRAM



- No hardcoded credentials
- Secure access with IAM roles
- Automatic rotation
- Works with Kubernetes
- Least privilege access
- Production ready

EXAMPLE KUBERNETES SECRET INJECTION

1. USING IRSA (RECOMMENDED)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: veriqta-app
spec:
  serviceAccountName: app-sa
  containers:
  - name: app
    image: veriqta-app:latest
    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
```

2. USING KUBERNETES SECRET

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  password: <your-secret>
---
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: app
    image: veriqta-app:latest
    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
```

KEEP SECRETS OUT OF CODE

PRODUCTION AWARENESS

- Never hardcode secrets in code or config
- Use AWS Secrets Manager for sensitive data
- Use IRSA instead of static AWS credentials
- Implement secret rotation and test it
- Grant least privilege access
- Monitor access and rotation events
- Handle secret retrieval failures gracefully
- Document secret management process
- Plan for key rotation and disaster recovery

VERIFICATION

- Application accesses secret successfully
- No credentials in code or container image
- Secret rotation works without downtime (or with minimal impact)
- IAM roles are correctly configured
- Application recovers after rotation
- Logs show no sensitive data
- Least privilege permissions are enforced
- You can troubleshoot access issues

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 16 OF 20

LEARN
BUILD
DEPLOY
GROW

FULL OBSERVABILITY STACK

OBSERVE
UNDERSTAND
TROUBLESHOOT
STAY AHEAD

Make the system observable before something breaks.

Prometheus
Metrics Collection

Grafana
Visualization & Dashboards

Logs
(e.g. Loki / ELK)

Jaeger
Distributed Tracing

Application & Infrastructure
Metrics, Logs, Traces

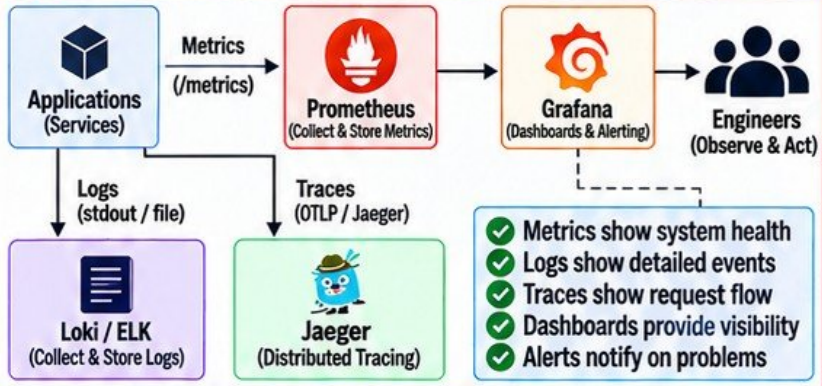
PROJECT OVERVIEW

In this project, you will implement a full observability stack using Prometheus and Grafana for metrics, a logging solution (e.g. Loki or ELK) for logs, and a tracing solution (e.g. Jaeger) for distributed tracing. You will learn to collect application and infrastructure metrics, create dashboards, set up alerting, and troubleshoot real issues by analyzing telemetry data. You will also simulate CPU or latency problems and identify them using the observability tools.

KEY LEARNING OBJECTIVES

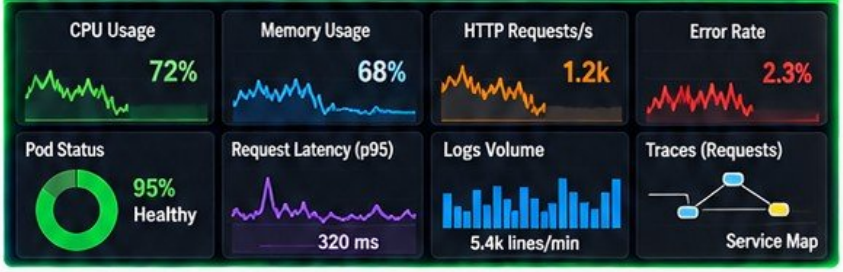
- Prometheus (metrics collection)
- Grafana (visualization and dashboards)
- Application metrics
- Infrastructure metrics (CPU, memory, disk, network)
- Logs (e.g. Loki / ELK)
- Dashboards
- Alerting (Prometheus Alertmanager)
- Basic distributed tracing (e.g. Jaeger)
- Failure lab: create CPU or latency problems and identify them from telemetry

OBSERVABILITY ARCHITECTURE



Logs + Metrics + Traces = Complete Observability

EXAMPLE GRAFANA DASHBOARD



IMPLEMENTATION STEPS

1. Deploy Prometheus (or use kube-prometheus)
2. Deploy Grafana and connect to Prometheus
3. Deploy a logging solution (Loki or ELK)
4. Deploy Jaeger for distributed tracing
5. Instrument a sample application (metrics, logs, tracee)
6. Create useful dashboards
7. Configure alerting rules and notifications
8. Simulate CPU or latency issues
9. Analyze telemetry data and identify the root cause
10. Fix the issue and verify from dashboards

EXAMPLE PROMETHEUS QUERY

```
# CPU usage by pod
rate(container_cpu_usage_seconds_total[5m])

# Memory usage by pod
container_memory_usage_bytes

# HTTP request rate
rate(http_requests_total[5m])

# Error rate
rate(http_requests_total{status=~"5.."}[5m])
```

FAILURE LAB

- Create CPU load (e.g. stress tool)
- Create latency (e.g. slow response)
- Observe the impact in Grafana
- Check logs for error messages
- Use tracing to find the slow service
- Identify the root cause from telemetry
- Fix the issue and verify recovery

PRODUCTION AWARENESS

- Collect key RED metrics (Rate, Errors, Duration)
- Monitor infrastructure and application metrics
- Centralize logs with proper retention
- Trace requests across services
- Set meaningful alerts (avoid alert noise)
- Make dashboards simple and actionable
- Test observability during incidents
- Plan for scaling, retention and cost

VERIFICATION

- Dashboards show live system metrics
- Logs are searchable and useful
- Traces show end-to-end request flow
- Alerts are triggered for issues
- You can identify CPU or latency problems from telemetry
- Root cause is found and fixed
- System returns to normal after the fix
- Observability helps you detect issues early

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Observability Turns Data into Action!

LEARN TODAY. BUILD TOMORROW. | VERIQTA

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 17 OF 20

LEARN
BUILD
DEPLOY
GROW

SRE MONITORING AND ALERTING PROJECT

OBSERVE
ALERT
RESPOND
IMPROVE

Turn monitoring into an operational reliability system.

PROJECT OVERVIEW

In this project, you will set up a complete monitoring and alerting system for a real application. You will learn how to define Service Level Indicators (SLIs), Service Level Objectives (SLOs), create dashboards, configure alert rules, write runbooks, track error budgets, and turn monitoring into an operational reliability system.

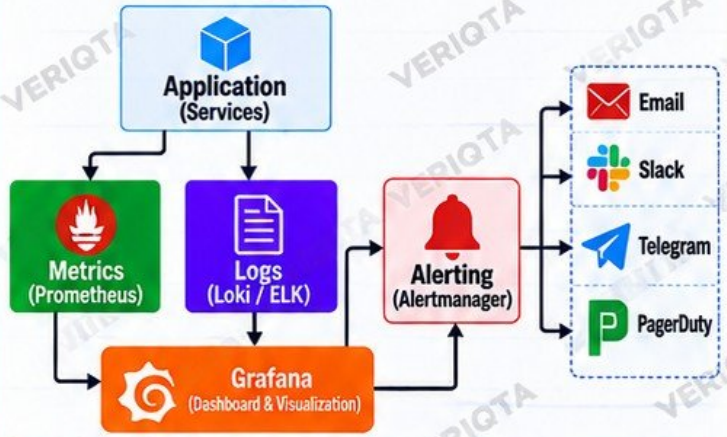
KEY LEARNING OBJECTIVES

- Service Level Indicators (SLIs)
- Service Level Objectives (SLOs)
- Availability
- Latency
- Error rates
- Saturation
- Alert rules
- Runbooks
- Error budgets
- Failure lab: trigger an actionable production alert

IMPLEMENTATION STEPS

1. Deploy application and generate traffic
2. Install and configure Prometheus (metrics)
3. Install and configure Grafana (dashboards)
4. Define Service Level Indicators (SLIs)
5. Define Service Level Objectives (SLOs)
6. Create alert rules in Alertmanager
7. Set up notifications (email, Slack or Telegram)
8. Write runbooks for common alerts
9. Test the monitoring and alerting system
10. Simulate failures and trigger an actionable alert

ARCHITECTURE DIAGRAM



From Metrics to Insights. From Alerts to Action. From Incidents to Reliability.

TECHNOLOGIES USED

Prometheus (Metrics) · Grafana (Dashboards) · Alertmanager (Alerts) · Loki (Logs) · Kubernetes (Optional)

EXAMPLE ALERT RULE (Prometheus)

```
alert: HighErrorRate
expr: rate(http_requests_total{status=~"5.."}[5m])
  / rate(http_requests_total[5m]) > 0.05
for: 2m
labels:
  severity: critical
annotations:
  summary: "High error rate detected"
  description: "Error rate is above 5% for more
  than 2 minutes. Investigate immediately."
```

Good Alerts
Reduce MTTR
Better Runbooks
Build Reliability

INTENTIONAL FAILURE LAB

- Increase application error rate (e.g. return 500)
- Trigger the alert and verify notification
- Check Grafana dashboard for the spike
- Follow the runbook to investigate
- Fix the issue and confirm alert is resolved

PRODUCTION AWARENESS

- Monitor what matters to the business
- Set realistic SLOs and error budgets
- Alert on symptoms, not just logs
- Avoid alert noise (tune thresholds)
- Have clear runbooks and ownership
- Monitoring is a proactive investment

VERIFICATION

- Dashboards show live metrics
- Alert triggers and notification is received
- Runbook is followed successfully
- Issue is identified and fixed
- Alert returns to normal state
- System is stable and within SLOs

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 18 OF 20

LEARN
BUILD
DEPLOY
GROW

DevSecOps
SOFTWARE SUPPLY CHAIN

Add security controls across delivery.

SECURE
BUILD
SAFE
DEPLOY
STRONGER
TOMORROW

PROJECT OVERVIEW

In this project, you will add security controls across the software delivery lifecycle. You will implement dependency scanning, SAST, secret scanning, container scanning, IaC scanning, generate a Software Bill of Materials (SBOM), configure CI/CD security gates and use least-privilege credentials for a secure pipeline.

KEY LEARNING OBJECTIVES

- Dependency scanning
- Static Application Security Testing (SAST)
- Secret scanning
- Container scanning
- Infrastructure as Code scanning
- Software Bill of Materials (SBOM)
- CI/CD security gates
- Least-privilege pipeline credentials
- Failure lab: introduce an exposed secret or vulnerable dependency

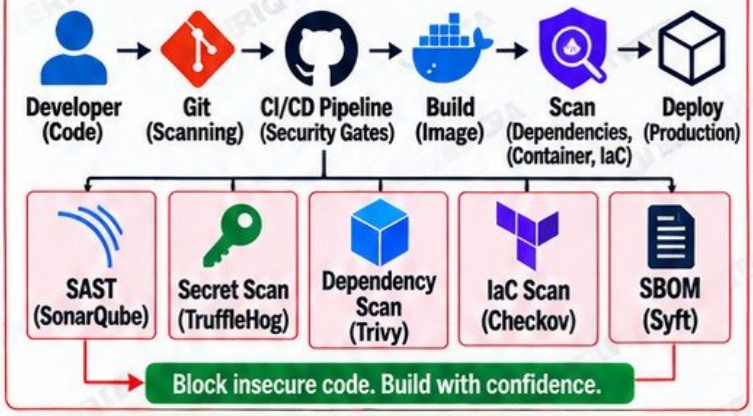
IMPLEMENTATION STEPS

1. Set up a sample application (e.g. Node.js or Python)
2. Add dependency scanning (e.g. Trivy, npm audit)
3. Implement SAST (e.g. SonarQube)
4. Configure secret scanning (e.g. TruffleHog)
5. Add container image scanning (e.g. Trivy/Grype)
6. Scan Infrastructure as Code (e.g. Checkov)
7. Generate and store SBOM (e.g. Syft)
8. Configure CI/CD security gates (fail on critical findings)
9. Use least-privilege pipeline credentials (e.g. IAM roles)

INTENTIONAL FAILURE LAB

- Add a vulnerable dependency (e.g. old lodash)
- Commit a fake API key or password in code
- Run the pipeline and observe security scan failure
- Check the report and identify the issue
- Fix the vulnerability/secret and re-run the pipeline

SOFTWARE SUPPLY CHAIN DIAGRAM



TECHNOLOGIES USED

Git · GitHub · SonarQube (SAST) · Trivy (Container Scan) · Checkov (IaC Scan)
Docker · Syft (SBOM) · Grype (Vulnerability Scan) · AWS (Cloud) · Kubernetes (Deployment)

EXAMPLE COMMAND (Trivy)

```
# Scan a container image for vulnerabilities
trivy image myapp:v1
# Scan IaC (Terraform) for misconfigurations
trivy config ./terraform
# Scan dependencies (Node.js)
trivy fs . --scanners vuln
# Generate SBOM
syft packages dir:. -o spdx-json > sbom.json
```

FIND
FIX
PREVENT
SHIP SAFELY

PRODUCTION AWARENESS

- Security is everyone's responsibility
- Fail fast on critical vulnerabilities
- Keep dependencies updated
- Use automated scanning in CI/CD
- Maintain a clear SBOM
- Use least-privilege access
- Prevent secrets from entering repositories
- Security controls should be enforceable

VERIFICATION

- Pipeline fails on vulnerable dependency
- Exposed secret is detected
- Scan reports are generated
- Pipeline blocks merge/deployment
- Issues are fixed and pipeline passes
- SBOM is created and stored
- Application is deployed securely
- Verify least-privilege credentials are used

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

SAME TOOLS
REAL ENGINEERS
REAL IMPACT

Security Builds Better Systems!

LEARN TODAY. BUILD TOMORROW. | VERIQTA

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 19 OF 20

LEARN
BUILD
DEPLOY
GROW

PRODUCTION INCIDENT RESPONSE LAB

Operate the environment as an on-call engineer.

PRACTICE
RESPOND
RESOLVE
LEARN
IMPROVE

PROJECT OVERVIEW

In this project, you will experience a realistic production incident from alert to resolution. You will act as an on-call engineer, investigate the issue, use logs and metrics, apply diagnostic commands, mitigate and recover the service, and perform a blameless postmortem.

KEY LEARNING OBJECTIVES

- Simulated outage
- Alert
- Triage
- Logs and metrics
- Hypothesis formation
- Diagnostic commands
- Mitigation
- Recovery
- Verification
- Root Cause Analysis
- Blameless postmortem
- Production lesson: restoring service and identifying root cause are separate objectives

INCIDENT RESPONSE FLOW

Alert (Detection) → Triage (Understand) → Logs & Metrics (Investigate) → Hypothesis (Form a theory) → Diagnostic Commands (Find the cause) → Mitigation (Fix the issue) → Recovery (Restore service) → Verification (Confirm it works) → Root Cause Analysis (Find the real cause) → Blameless Postmortem (Learn and improve)

Restoring service and identifying root cause are separate objectives.

TOOLS & TECHNOLOGIES (EXAMPLES)

- Linux (Commands)
- Prometheus (Metrics)
- Grafana (Visualization)
- ELK Stack (Logs)
- CloudWatch (Metrics & Logs)
- Kubernetes (Platform)
- AWS (Cloud)
- Docker (Containers)
- Slack (Alerts)
- PagerDuty (On-Call)

IMPLEMENTATION STEPS

1. Set up a sample application in a cloud or Kubernetes environment
2. Configure monitoring and alerts
3. Simulate a realistic outage (e.g, stop a service, high CPU, or DB failure)
4. Receive and triage the alert
5. Investigate using logs, metrics and diagnostic commands
6. Form a hypothesis and identify the root cause
7. Apply mitigation steps
8. Restore the service
9. Verify the service is fully recovered
10. Perform a blameless postmortem and document learnings

EXAMPLE DIAGNOSTIC COMMANDS (LINUX)

```
# Check running services
sudo systemctl status <service>
# Check logs
sudo journalctl -u <service> -n 100 --no-pager
# Check processes
ps aux | grep <service>
# Check network connectivity
curl -I http://<service-endpoint>
# Check resource usage
top
df -h
# Check Kubernetes resources (if using k8s)
kubectl get pods
kubectl describe pod <pod-name>
```

INVESTIGATE
MITIGATE
RECOVER
LEARN
BE BETTER

INTENTIONAL FAILURE LAB

- Simulate an outage (e.g. stop the application or database)
- Observe the alert and respond as on-call
- Investigate and identify the root cause
- Apply a fix and restore the service
- Verify recovery
- Document findings in a postmortem

PRODUCTION AWARENESS

- Incidents will happen in production
- Stay calm and follow a process
- Focus on restoring service first
- Use data, not assumptions
- Communicate clearly with stakeholders
- Learn from every incident
- Build and maintain runbooks
- Blameless culture leads to stronger teams

VERIFICATION

- Alert is triggered and received
- Root cause is correctly identified
- Service is successfully restored
- Application is fully functional
- Postmortem is documented
- Action items are created
- Monitoring and alerts are improved
- Runbooks are updated

20 PRODUCTION-GRADE DEVOPS PROJECTS
REAL PROJECTS. REAL SKILLS. REAL OPPORTUNITIES.

VERIQTA

PROJECT 20 OF 20

LEARN
BUILD
DEPLOY
GROW

CAPSTONE: PRODUCTION DEVOPS PLATFORM

Combine the entire curriculum into one production-style system.

REAL SKILLS
REAL SYSTEMS
REAL IMPACT

PROJECT OVERVIEW

In this capstone project, you will design, build and operate a complete, production-style DevOps platform. You will use infrastructure as code, containerization, CI/CD, GitOps, security controls, observability, and incident response to deliver a real application on AWS EKS.

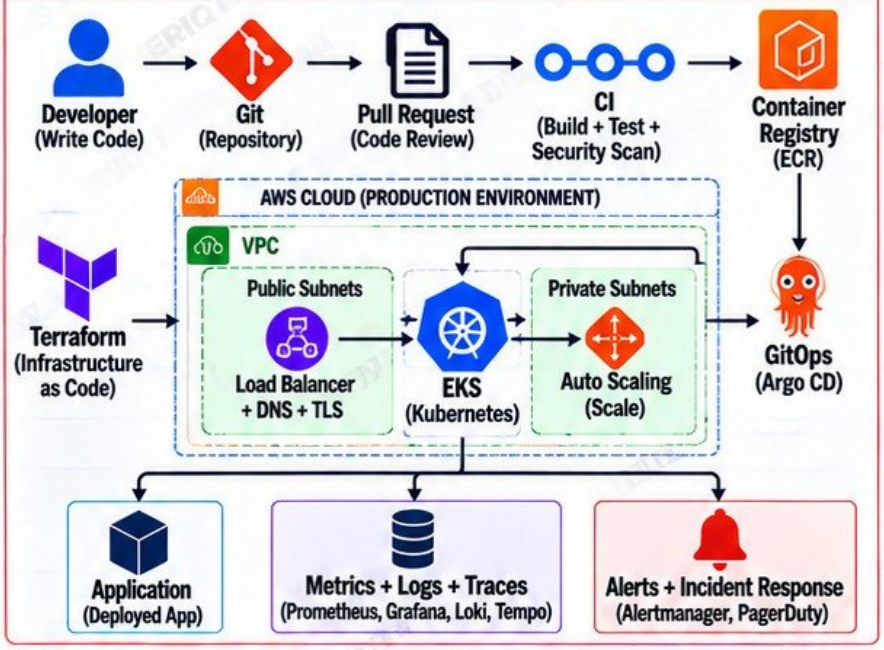
KEY LEARNING OBJECTIVES

- End-to-end DevOps platform design
- Infrastructure as code with Terraform
- Secure AWS networking
- Kubernetes (EKS) and GitOps
- Docker and container registry
- CI/CD with testing and security scans
- Secrets management
- Observability (metrics, logs, traces)
- Autoscaling and deployment strategy
- Rollback and incident response
- Documentation and runbooks

IMPLEMENTATION COMPONENTS

1. Terraform infrastructure (VPC, subnets, IAM, EKS)
2. Secure AWS networking and security groups
3. Dockerize the application
4. CI/CD pipeline (build, test, security scan)
5. Store images in Amazon ECR
6. GitOps deployment (Argo CD)
7. Secrets management (AWS Secrets Manager)
8. Deploy application to EKS
9. Configure observability (Prometheus, Grafana, Loki, Tempo)
10. Set up autoscaling and deployment strategy (rolling/canary)
11. Implement rollback process
12. Create documentation, runbooks and simulate a production incident

END-TO-END ARCHITECTURE DIAGRAM



DEPLOYMENT FLOW

Code → Build → Test → Scan → Deploy → Monitor

Automate. Secure. Deploy. Observe. Respond. Improve.

TECHNOLOGIES USED

Terraform (IaC) · AWS (Cloud) · Kubernetes (EKS) · Docker (Containers) · Argo CD (GitOps) · GitHub (Source Control)

Prometheus (Metrics) · Grafana (Visualization) · Loki (Logs) · Tempo (Traces) · AWS Secrets Manager (Secrets) · PagerDuty (Incident Response)

SIMULATED PRODUCTION INCIDENT

- Introduce a failure (e.g, remove pods, high CPU, or bad deployment)
- Detect the issue through alerts
- Triage using logs, metrics and traces
- Apply mitigation (scale, rollback or fix)
- Restore service
- Perform root cause analysis
- Write a blameless postmortem

PRODUCTION AWARENESS

- Security is built in from the start
- Infrastructure is reproducible
- Everything is version controlled
- Monitor what matters
- Be ready for failures
- Automate recovery where possible
- Document and share knowledge
- Real systems have real trade-offs

SUCCESS CRITERIA

- Application is publicly accessible
- CI/CD pipeline is automated and secure
- Infrastructure is created with Terraform
- Application is deployed via GitOps
- Observability and alerting are working
- Autoscaling is configured
- Security controls are in place
- Runbooks and documentation are completed
- Incident is simulated and resolved

VERIQTA

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Ideas Skills Systems A Better Tomorrow!

LEARN TODAY. BUILD TOMORROW. | VERIQTA